Production-Grade Spring Boot Microservices Learning Project

Recommended project: Enterprise E-Commerce & Order Management Platform. The goal is to learn Spring Boot microservices end-to-end through one realistic distributed system.

1. Overall Architecture

Client → API Gateway → User / Product / Order / Cart Services → Databases. Kafka connects asynchronous workflows such as Payment, Inventory, Notification and Shipping. Observability is provided through Prometheus, Grafana, OpenTelemetry and centralized logging. Docker and Kubernetes are used for deployment.

2. Core Microservices

Service	Purpose and Learning
User Service	Registration, login, profiles, roles and addresses. Learn Spring Data JPA, PostgreSQL, Spring Security, JWT, DTOs, validation and exception handling.
Product Service	Products, categories, brands, pricing and search. Learn CRUD, pagination, filtering, indexing, optimistic locking and caching.
Cart Service	Add/remove products, quantity updates and cart lifecycle. Use Redis to learn caching, TTL and cache invalidation.
Order Service	Create/cancel orders, order history and status management. Use this as the main business service to learn transactions, state machines and Saga.
Payment Service	Simulated payment processing. Learn retries, timeouts, idempotency and failure handling.
Inventory Service	Stock checks, reservations, releases and deductions. Learn concurrency, optimistic/pessimistic locking and consistency.
Notification Service	Email, SMS and push notifications using asynchronous Kafka events.
Shipping Service	Shipment creation, tracking and delivery status.
Review Service	Product ratings and comments using MongoDB to learn polyglot persistence.

3. API Gateway and Service Discovery

Use Spring Cloud Gateway for routing, authentication, authorization, rate limiting, CORS and circuit-breaker integration. Start with Eureka for service discovery, then move to Kubernetes-native service discovery to understand why Eureka is often unnecessary in Kubernetes environments.

4. Configuration Management

Use Spring Cloud Config to externalize database, Kafka, Redis, JWT and external-service configuration. Maintain environment-specific configuration such as application-dev.yml, application-test.yml and application-prod.yml. Handle secrets separately from ordinary configuration.

5. Kafka and Event-Driven Architecture

Create topics such as order.created, order.confirmed, order.cancelled, payment.success, payment.failed, inventory.reserved, inventory.failed, shipment.created and shipment.delivered.

Use synchronous REST calls when an immediate response is required and Kafka for asynchronous workflows and decoupling. Learn producers, consumers, consumer groups, partitions, offsets, retries and dead-letter topics.

6. Saga Pattern

Implement both Saga choreography and Saga orchestration. A typical flow is: Create Order → Reserve Inventory → Process Payment → Confirm Order. If payment fails, compensate by releasing inventory and cancelling the order. This teaches distributed transactions and eventual consistency.

7. Transactional Outbox Pattern

Solve the dual-write problem where a database transaction succeeds but Kafka publication fails. Store the business record and an outbox event in the same database transaction, then use an outbox publisher to deliver events to Kafka. Learn at-least-once delivery, duplicate events and idempotent consumers.

8. Resilience

Use Resilience4j to implement Circuit Breaker, Retry, Timeout, Rate Limiting, Bulkhead and Fallback. Test failure scenarios including Payment Service downtime, slow services, Kafka failures, database failures and Redis failures.

9. Security

Implement Spring Security with JWT and OAuth2 concepts. Add role-based access control for CUSTOMER, ADMIN, SELLER and WAREHOUSE. Later integrate Keycloak to gain practical experience with centralized identity and access management.

10. Observability

Use Spring Boot Actuator and Micrometer for application metrics, Prometheus for metrics collection, Grafana for dashboards, OpenTelemetry for distributed tracing, Jaeger/Tempo for trace visualization, and ELK/OpenSearch for centralized logs. Ensure a single traceId can follow a request from API Gateway through Order, Inventory and Payment services.

11. Testing Strategy

Implement multiple testing layers: JUnit 5 and Mockito for unit tests; Spring Boot Test and Testcontainers for integration tests; Spring Cloud Contract for contract testing; Postman and REST Assured for API testing. Use Testcontainers for PostgreSQL, Kafka, Redis and MongoDB.

12. Docker and Kubernetes

Containerize each service and supporting infrastructure. Use Docker Compose for local development. Then deploy to Kubernetes using Pods, Deployments, Services, ConfigMaps, Secrets, Ingress, Namespaces, HPA, PersistentVolumes and StatefulSets. Use Helm for repeatable deployments and demonstrate horizontal scaling under load.

13. CI/CD

Build a GitHub Actions pipeline that performs compilation, unit tests, integration tests, static analysis with SonarQube, security scanning with Trivy, Docker image creation, image publishing and Kubernetes deployment.

14. Recommended Technology Stack

Area	Technology
Language	Java 21
Framework	Spring Boot
API	REST
Gateway	Spring Cloud Gateway
Discovery	Eureka → Kubernetes
Config	Spring Cloud Config
Security	Spring Security + JWT + OAuth2
Identity	Keycloak
Database	PostgreSQL
NoSQL	MongoDB
Cache	Redis
Search	Elasticsearch / OpenSearch
Messaging	Apache Kafka
Resilience	Resilience4j
Distributed Transactions	Saga Pattern
Reliability	Transactional Outbox
Testing	JUnit + Mockito + Testcontainers
Contract Testing	Spring Cloud Contract
Containers	Docker
Orchestration	Kubernetes
Deployment	Helm
Metrics	Prometheus
Dashboard	Grafana
Tracing	OpenTelemetry
Logs	ELK / OpenSearch
CI/CD	GitHub Actions
Cloud	AWS

15. Recommended Learning Sequence

Stage 1 — Foundation

Spring Boot, REST, JPA, PostgreSQL, exception handling, validation and JUnit. Build User and Product services.

Stage 2 — Microservices

Build Order, Payment and Inventory services. Learn OpenFeign, REST communication, Eureka, API Gateway and Config Server.

Stage 3 — Distributed Systems

Kafka, Saga, Outbox, Idempotency, Eventual Consistency and Distributed Transactions.

Stage 4 — Resilience

Circuit Breaker, Retry, Timeout, Bulkhead and Rate Limiting.

Stage 5 — Data

PostgreSQL, MongoDB, Redis and Elasticsearch/OpenSearch. Learn database-per-service and polyglot persistence.

Stage 6 — Security

Spring Security, JWT, OAuth2, Keycloak and RBAC.

Stage 7 — Production Readiness

Docker, Kubernetes, Helm, Prometheus, Grafana, OpenTelemetry and centralized logging.

Stage 8 — DevOps

GitHub Actions, CI/CD, SonarQube, Trivy, container registry and AWS.

16. Final Learning Roadmap

```
REST → Microservices → Gateway → Discovery → Kafka → Saga → Outbox → Idempotency →
Resilience → Security → Observability → Docker → Kubernetes → CI/CD → AWS
```

17. Final Recommendation

Given your existing Java, Spring Boot, Kafka, Docker, Kubernetes and microservices background, avoid spending months on basic CRUD applications. Build this as a production-grade distributed system and deliberately implement one microservices concept at a time. The project can become both a strong portfolio project and a practical preparation platform for Senior Software Engineer, Staff Engineer and Tech Lead interviews.